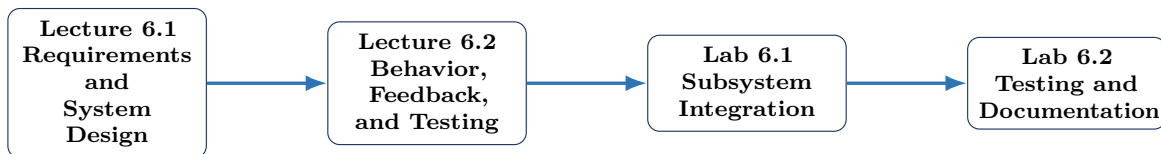


Week 6: Arduino Project Design and Integration

Lecture 6.1 and Lecture 6.2

ENGR 1105 | Fall 2026



Week 6 Goal

Turn separate sensors, controller logic, outputs, actuators, and power connections into one reliable Arduino project. The project should have clear requirements, safe behavior, testable subsystems, documented interfaces, and evidence that it meets its intended purpose.

Meeting	Main focus
Lecture 6.1	Requirements, block diagrams, subsystem interfaces, pin planning, power, mechanical layout, and integration strategy
Lecture 6.2	Feedback, thresholds, state-based logic, timing, fault behavior, testing, reliability, safety, and troubleshooting
Lab 6.1	Assemble and test sensors, controller logic, outputs, and actuators as separate subsystems
Lab 6.2	Complete integration, system testing, troubleshooting, documentation, and final-lab submission

Opening Questions

1. What can go wrong when two working subsystems are connected together?
2. How can a requirement be written so that another person can test it?
3. What should a project do when a sensor reading is missing or unreasonable?

Lecture 6.1

Arduino Project Design and Subsystem Integration

Monday, November 30, 2026

Learning Objectives

By the end of Lecture 6.1, you should be able to:

1. translate a project idea into measurable requirements and constraints;
2. create a block diagram that separates sensing, control, power, and physical output;
3. define subsystem interfaces using signals, units, voltage levels, and pin assignments;
4. estimate an initial current and power budget;
5. plan the mechanical layout and wiring before final assembly; and
6. use incremental integration to reduce troubleshooting time.

1. From Idea to Engineering Project

A project idea describes a desired experience. An engineering project also states what the system must do, how success will be measured, and what limits must be respected.

Requirement

A **requirement** is a clear, testable statement describing a behavior, performance level, interface, safety condition, or constraint that the project must satisfy.

Weak statement	Improved testable requirement
The robot should avoid objects.	When a valid distance is below 20 cm, the drive motor shall stop within 0.5 s.
The light should be bright.	In the warning state, the LED shall operate at PWM 180 or higher.
The arm should move safely.	The servo command shall remain between 20° and 160°.

A Requirement Must Be Testable

Words such as “good,” “fast,” or “safe” are not enough by themselves. Add a measurable condition, threshold, time, range, or observable result.

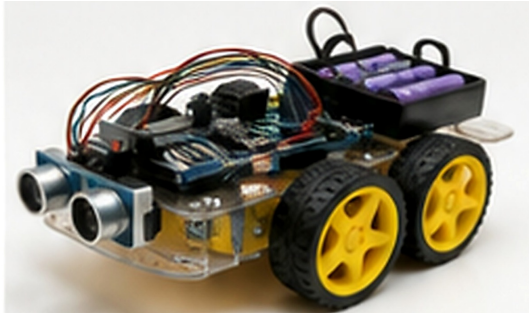
2. Constraints

Constraints limit the design. Common constraints include:

- available Arduino pins;
- actuator voltage and current;
- available sensors and mechanical parts;
- project size and operating space;
- limited build and test time;
- safety rules;
- required demonstration behavior.

3. Real Project Examples

The photographs below show actual physical systems that combine structure, sensors or inputs, controller logic, power, and actuators.



Arduino robot car: distance sensing, controller, motor driver, battery, wheels, and chassis.

Actual photograph reused from the instructor's earlier ENGR 1105 media library.



Automatic door: presence sensing, state logic, motor drive, and safety behavior.

Actual photograph reused from the instructor's earlier ENGR 1105 media library.



Robotic arm: several actuators, mechanical limits, coordinated commands, and a gripper.

Actual photograph reused from the instructor's earlier ENGR 1105 media library.



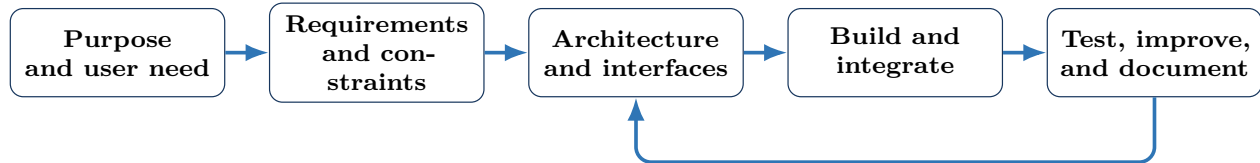
Robotic vacuum: sensors, drive motors, navigation behavior, battery, and fault handling.

Actual photograph reused from the instructor's earlier ENGR 1105 media library.

Subsystem Detective

Choose one photograph. Identify its likely input, controller, actuator, power source, mechanical structure, normal operating state, and one possible fault state.

4. The Engineering Design Flow



Design Is Iterative

Testing may reveal that a requirement is unclear, an interface is incorrect, or a subsystem needs revision. Returning to an earlier step is normal engineering work.

5. Define the Project Purpose

A useful purpose statement includes:

- the user or situation;
- the desired physical behavior;
- the important input or sensor;
- the main output or actuator.

Purpose Statement

The project will help a user maintain a safe distance from an obstacle by measuring distance with an ultrasonic sensor and commanding a motor and warning LED.

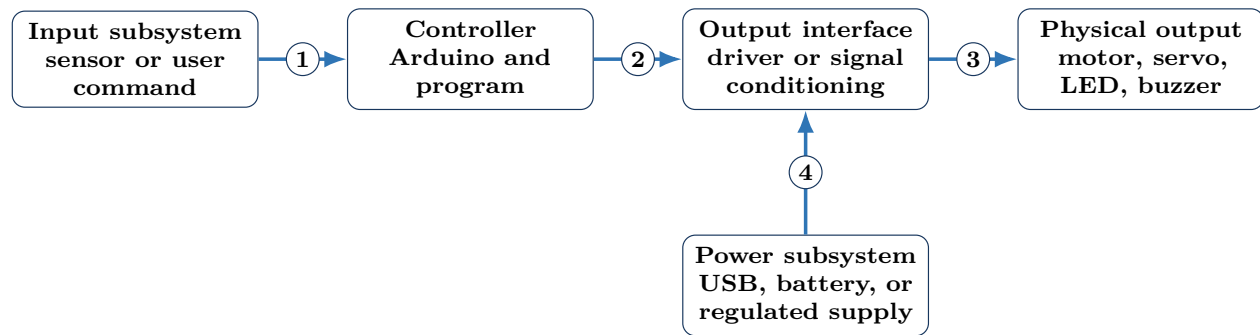
6. Define Success Criteria

Success criteria should answer:

- What behavior must be demonstrated?
- What range, threshold, or response time matters?
- What invalid or unsafe condition must be handled?
- What evidence will be recorded?

7. System Block Diagram

A block diagram shows the major subsystems and the direction of information or power. It is not a detailed wiring diagram.



1 measurement or user input

2 logic-level command

3 actuator power or output signal

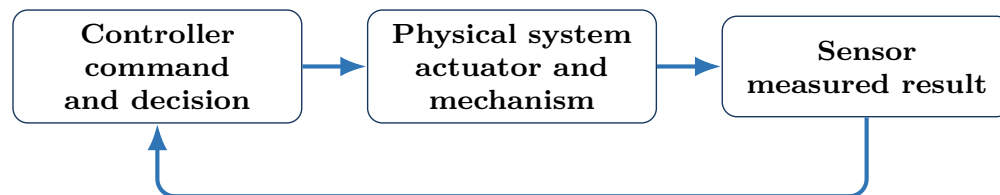
4 power input

Information and Energy Are Different

A signal tells a subsystem what to do. A power path provides the energy needed to do it. A small Arduino signal may control a much larger motor current through a driver.

8. Add Feedback When Needed

A feedback arrow represents a measured result returning to the controller.



9. Subsystem Interfaces

An interface is the boundary where two subsystems exchange information, power, or physical motion.

Interface	From/to	What crosses boundary	Questions to answer
Sensor signal	sensor to Arduino	voltage, digital state, or timed pulse	pin, range, units, update rate, invalid value
Control signal	Arduino to driver or servo	HIGH/LOW, PWM, or pulse command	voltage level, pin, safe startup value
Power	supply to electronics or actuator	voltage and current	polarity, current capability, common ground
Mechanical	actuator to load	force, torque, speed, or position	attachment, range, interference, overload

10. Create an Interface Table

Robot-Car Interface Table

Connection	Signal or power	Arduino pin	Expected behavior
Ultrasonic TRIG	digital output	D7	10 μ s trigger pulse
Ultrasonic ECHO	timed digital input	D6	valid pulse or timeout
Motor PWM	PWM output	D9	0 means stopped
Driver direction	digital outputs	D4, D5	selected forward or reverse state
Motor supply	external power	not an I/O pin	approved voltage and current

11. Create an I/O and Pin Budget

Device	Inputs needed	Outputs needed	Special requirement
HC-SR04	1 digital	1 digital	timed pulse measurement
Potentiometer	1 analog	0	<code>analogRead()</code>
Servo	0	1 digital	servo control pulse
DC driver	0	2 digital + 1 PWM	external motor power
LED	0	1 digital or PWM	series resistor

12. Power Budget

A first power budget estimates whether each supply can support the connected loads.

Load	Typical estimate	Worst-case estimate	Supply path
Arduino and sensors	80 mA	150 mA	USB or regulated logic supply
Servo	150 mA	700 mA	approved servo supply
DC motor	250 mA	1.2 A stall	motor supply through driver
LED and buzzer	20 mA	40 mA	logic or output supply

For loads on one supply, an initial estimate is

$$I_{\text{required}} \approx I_1 + I_2 + \cdots + I_n.$$

Current Estimate

Suppose the motor supply powers two motors that may each draw 1.2 A at stall and a driver using approximately 0.1 A.

$$I_{\text{worst case}} = 2(1.2 \text{ A}) + 0.1 \text{ A} = 2.5 \text{ A}.$$

The supply and driver must be selected using the actual component ratings and the expected operating condition.

Do Not Guess Motor Current

Motor current changes with startup, load, and stall. Use the component specification or an instructor-approved estimate, and stop the system if a motor or driver becomes hot.

13. Common Ground

When an Arduino commands a separately powered driver, the logic grounds usually need a shared reference.

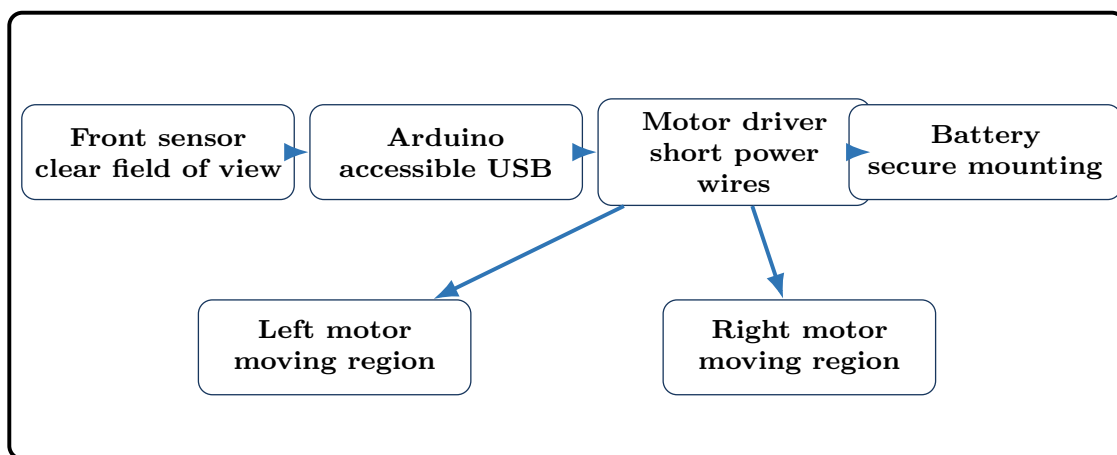
$$\text{Arduino GND} \longleftrightarrow \text{driver logic GND} \longleftrightarrow \text{motor-supply negative}.$$

Common ground does not mean motor current should flow through an Arduino I/O pin.

14. Mechanical Layout and Wiring Plan

Before final assembly, sketch:

- where sensors face and what they can detect;
- where actuators and moving parts travel;
- where the Arduino, driver, and battery will be mounted;
- how wires will avoid wheels, horns, gears, and linkages;
- where USB access and the power switch remain reachable.



Layout Is Part of System Design

A circuit can be electrically correct but mechanically unreliable if wires are loose, sensors are blocked, the battery shifts, or a moving part strikes another component.

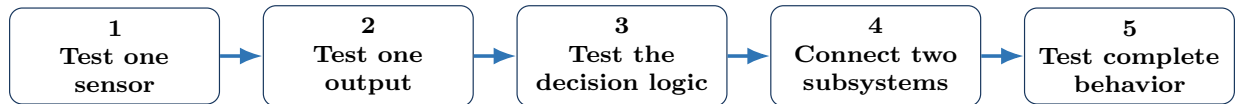
15. Wire Management

Useful practices:

- keep power wires short and secure;
- separate noisy motor wiring from sensitive sensor wiring when possible;
- label repeated connections;
- provide strain relief;
- leave service loops only where movement requires them;
- avoid unsupported connections that can pull out.

16. Incremental Integration

Do not build the complete project and test everything for the first time at once.



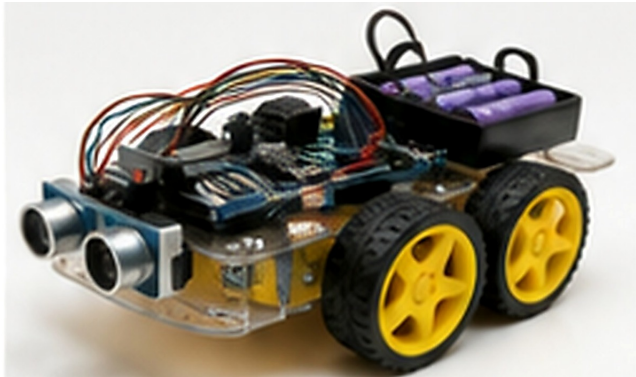
Know the Last Working Stage

Incremental integration creates a known-good baseline. When a new connection causes a failure, the search area is smaller.

17. Recommended Bring-Up Sequence

1. Verify power polarity with the actuator disconnected.
2. Upload code that begins in a stopped or safe state.
3. Test sensor readings in the Serial Monitor.
4. Test the output or actuator independently.
5. Print the calculated state or command.
6. Connect sensor behavior to the output at a low-risk command.
7. Add filtering, timing, and fault handling.
8. Test limits and failure conditions.
9. Secure the mechanical layout only after behavior is stable.

18. Real Example: Robot-Car Architecture



Actual photograph reused from the instructor’s earlier ENGR 1105 media library.

Subsystem	Likely role
Ultrasonic sensor	measure obstacle distance
Arduino	calculate state and commands
Motor driver	switch motor current
Battery	provide motor energy
Chassis and wheels	support and transmit motion
Program	choose drive, slow, turn, or stop behavior

19. Project Design Document

Before Lab 6, each project should have:

Required item	Minimum content
Purpose	one clear sentence describing user, input, and physical output
Requirements	three to five testable behavior or safety statements
Block diagram	sensing, controller, interface, actuator/output, and power
Pin table	every Arduino pin and connected component
Power plan	voltage source, current path, common ground, and safe startup
State plan	normal states, transitions, and fault response
Test plan	test condition, expected result, observed result, and pass/fail rule

Lecture 6.1 Design Review

For your final Arduino project:

1. write one purpose statement;
2. write three testable requirements;
3. list the sensor, controller, output interface, actuator, and power source;
4. identify the subsystem you will test first.

Lecture 6.1 Exit Questions

1. Why is a block diagram different from a wiring diagram?
2. What information belongs in an interface table?
3. Why should startup and stall current be considered in a power budget?
4. Why does incremental integration reduce troubleshooting time?

Lecture 6.2

Project Behavior, Feedback, Testing, and Reliability

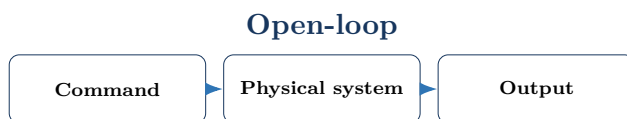
Tuesday, December 1, 2026

Learning Objectives

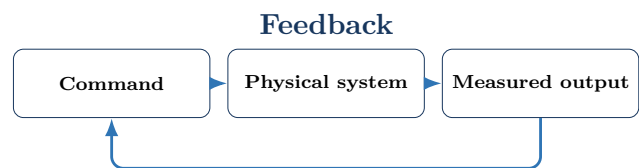
By the end of Lecture 6.2, you should be able to:

1. distinguish open-loop behavior from feedback behavior;
2. organize project behavior into named states and transitions;
3. use thresholds, hysteresis, timeouts, and debouncing;
4. use `millis()` for nonblocking timing;
5. define fault states and safe default outputs;
6. create a subsystem and system test matrix; and
7. troubleshoot using evidence rather than random changes.

20. Open-Loop and Feedback Behavior



The controller does not measure whether the output achieved the desired result.



A sensor measures the result, and the controller uses that information in the next decision.

Examples

- Open loop: command a DC motor with PWM 150 without measuring speed.
- Feedback: measure obstacle distance and stop the motor when distance becomes too small.
- Internal feedback: a hobby servo measures its shaft position inside the servo.

21. State-Based Logic

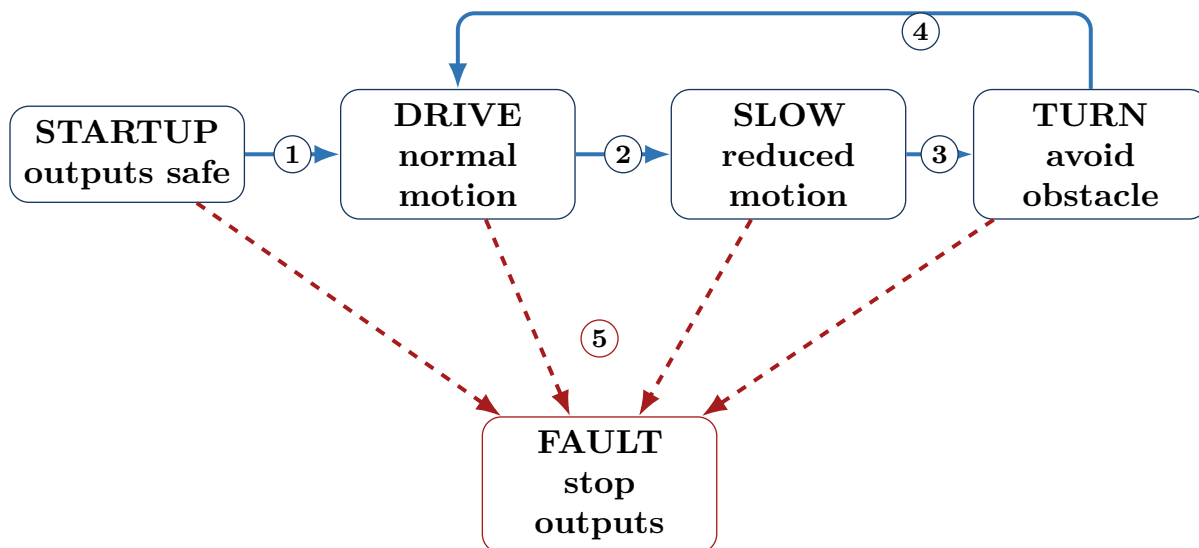
A state is a named operating condition with a defined set of outputs.

State Machine

A **state machine** organizes behavior into named states. A transition occurs when an event, condition, timeout, or fault changes the active state.

State	Output behavior	Example transition
STARTUP	motors stopped; check sensor	valid sensor reading received
DRIVE	motor runs at normal command	distance falls below warning threshold
SLOW	reduced motor command	distance becomes safe again
TURN	left and right motors differ	turn timer expires
FAULT	actuators stopped; warning active	reset after problem is corrected

22. State Diagram Example



- 1 sensor valid 2 distance below warning limit 3 distance below danger limit 4 turn time completed
5 timeout, invalid data, or unsafe condition

Outputs Depend on the Active State

Instead of scattering many unrelated `if` statements through the program, define the desired outputs for each state and the conditions that cause transitions.

23. State Logic in Arduino Code

```
1  enum RobotState {
2      STARTUP,
3      DRIVE,
4      SLOW,
5      TURN,
6      FAULT
7  };
8
9  RobotState state = STARTUP;
10
11 void loop() {
12     float distanceCm = readDistanceCm();
13
14     switch (state) {
15         case STARTUP:
16             motorStop();
17
18             if (distanceCm > 0) {
19                 state = DRIVE;
20             }
21             break;
22
23         case DRIVE:
24             motorForward(170);
25
26             if (distanceCm < 0) {
27                 state = FAULT;
28             }
29             else if (distanceCm < 35) {
30                 state = SLOW;
31             }
32             break;
33
34         case SLOW:
35             motorForward(100);
36
37             if (distanceCm < 0) {
38                 state = FAULT;
39             }
40             else if (distanceCm < 18) {
41                 state = TURN;
42             }
43             else if (distanceCm > 42) {
44                 state = DRIVE;
45             }
46             break;
47     }
48 }
```

Continue the switch statement with the TURN and FAULT behavior.

Use Safe Outputs First

At startup and in a fault state, actuators should normally be stopped or moved to a known safe position before the program attempts normal behavior.

24. Thresholds and Hysteresis

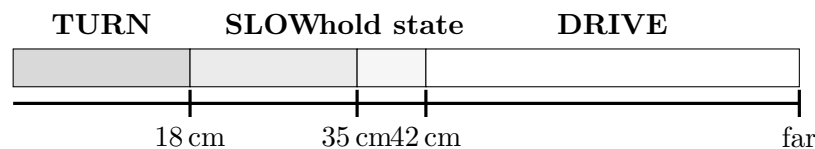
A single threshold can cause rapid switching when measurements vary near the boundary.

Example:

34.8, 35.2, 34.9, 35.1 cm.

Use separate transition limits:

- enter the SLOW state below 35 cm;
- return to DRIVE above 42 cm.



25. Debouncing

A mechanical button may rapidly change between HIGH and LOW during one press. Debouncing requires the input to remain stable for a short time before the program accepts the change.

```

1  const unsigned long debounceMs = 40;
2
3  if (reading != lastReading) {
4      lastChangeTime = millis();
5  }
6
7  if (millis() - lastChangeTime > debounceMs) {
8      stableButton = reading;
9  }
10
11 lastReading = reading;
```

26. Timeouts and Invalid Data

A sensor function should distinguish a valid measurement from missing data.

```
1 float readDistanceCm() {
2     unsigned long duration =
3         pulseIn(echoPin, HIGH, 30000);
4
5     if (duration == 0) {
6         return -1.0;
7     }
8
9     float distance =
10         duration * 0.0343 / 2.0;
11
12     if (distance < 2.0 ||
13         distance > 400.0) {
14         return -1.0;
15     }
16
17     return distance;
18 }
```

Invalid Does Not Mean Zero Distance

A timeout value should not be treated as an object at 0 cm. Create a separate invalid-data response, such as holding the previous safe state or entering FAULT.

27. Repeated Confirmation

For important transitions, require several consecutive readings.

```
1 if (distanceCm > 0 &&
2     distanceCm < 18.0) {
3     closeCount++;
4 } else {
5     closeCount = 0;
6 }
7
8 if (closeCount >= 3) {
9     state = TURN;
10 }
```

The benefit is fewer false transitions. The cost is a small response delay.

28. Nonblocking Timing with millis()

A long `delay()` prevents the program from reading sensors or responding to faults during the delay.

```

1 unsigned long turnStart = 0;
2 const unsigned long turnTime = 700;
3
4 void startTurn() {
5     state = TURN;
6     turnStart = millis();
7 }
8
9 void updateTurn() {
10    motorTurnRight(130);
11
12    if (millis() - turnStart >= turnTime) {
13        motorStop();
14        state = DRIVE;
15    }
16 }
```

Method	Main advantage	Main limitation
<code>delay()</code>	simple for an early isolated test	program cannot perform other work during delay
<code>millis()</code>	sensors, outputs, and states can be updated repeatedly	requires time variables and state logic

Responsive Projects Update Repeatedly

A useful loop reads inputs, validates data, updates the state, commands outputs, and repeats without unnecessary blocking.

29. Recommended Loop Structure

```

1 void loop() {
2     readInputs();
3     validateInputs();
4     updateState();
5     commandOutputs();
6     reportStatus();
7 }
```

Each function has one clear responsibility.

30. Fault States and Safe Defaults

A fault state should define:

- what condition caused the fault;
- which outputs are stopped or limited;
- how the user is informed;
- whether automatic recovery is allowed;
- what reset or correction is required.

Fault condition	Safe response	Recovery rule
Sensor timeout	stop motor; turn on fault LED	require three valid readings
Motor command active too long	stop output	user reset after inspection
Servo command outside safe range	constrain command	return to allowed range
Battery voltage too low	reduce load or stop	recharge or replace battery

```

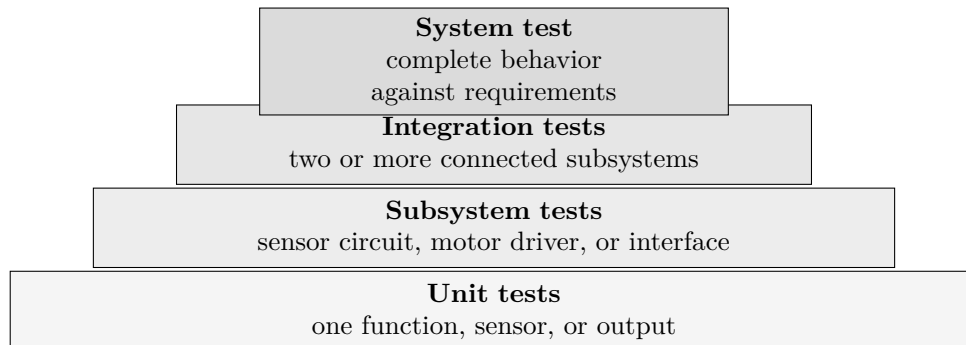
1 case FAULT:
2   motorStop();
3   digitalWrite(faultLed, HIGH);
4
5   if (validCount >= 3 &&
6       resetButtonPressed()) {
7     digitalWrite(faultLed, LOW);
8     state = STARTUP;
9   }
10  break;

```

Do Not Hide a Fault with Repeated Resets

If the Arduino repeatedly resets when a motor starts, investigate power, grounding, current capability, and wiring. Adding delays or immediately restarting the motor does not correct the electrical problem.

31. Levels of Testing



Start Low in the Test Structure

When a system test fails, return to subsystem or unit tests. Confirm the simplest pieces before changing the entire project.

32. A Test Has Four Parts

1. **Condition:** what input, setup, or starting state is used?
2. **Expected result:** what should happen?
3. **Observed result:** what actually happened?
4. **Decision:** pass, fail, or revise?

Requirement-Based Test

Requirement: When distance is below 20 cm, the drive motor shall stop within 0.5 s.

Test condition: Move a flat target from 40 cm to 15 cm while the motor is running.

Evidence: Record distance, state, motor command, and response time.

Pass rule: Motor command becomes zero within 0.5 s for three repeated trials.

33. Test Matrix

Test	Condition	Expected result	Observed result	Pass?
T1	normal safe distance	DRIVE state; motor 170		
T2	warning distance	SLOW state; motor 100		
T3	danger distance	TURN state; timed turn		
T4	no echo / timeout	FAULT; motor stopped		
T5	power cycle	STARTUP with safe outputs		

34. Test Boundaries, Not Only Typical Values

For a threshold of 35 cm, test:

34, 35, 36 cm

and several repeated measurements near the threshold.

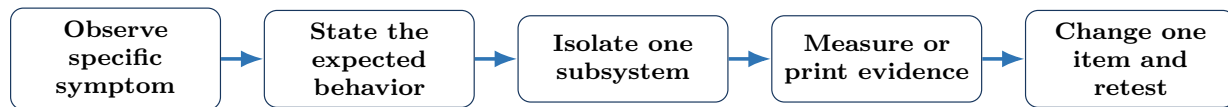
Also test:

- minimum and maximum commands;
- startup and power-cycle behavior;
- disconnected or invalid sensor data;
- a partially blocked mechanism;
- low battery or motor startup disturbance, when safely supervised.

A Successful Demonstration Is Not a Complete Test

A system may work once under ideal conditions and fail near a threshold, after a power cycle, or when a cable moves. Reliability requires repeated and boundary testing.

35. Troubleshooting Workflow



Evidence Before Changes

Use Serial output, LEDs, voltage measurements, continuity checks, or simple test code to determine what the system is doing. Randomly changing several items destroys useful evidence.

36. Troubleshooting by Layer

Layer	First checks
Power	correct voltage, polarity, current capability, common ground, loose connection
Sensor	wiring, pin number, useful range, timeout, physical placement
Logic	printed input value, active state, threshold order, variable type
Output interface	enable pins, direction pins, PWM-capable pin, driver rating
Mechanical	friction, interference, load, loose wheel or horn, blocked motion

37. Serial Diagnostic Output

```

1 Serial.print("distance=");
2 Serial.print(distanceCm);
3
4 Serial.print(", state=");
5 Serial.print(state);
6
7 Serial.print(", motor=");
8 Serial.println(motorCommand);
  
```

A useful diagnostic line connects the measured input, active decision, and output command.

38. Reliability and Robustness

A robust project behaves predictably when conditions vary.

Technique	Reliability benefit
Input validation	prevents impossible or missing data from controlling outputs
Filtering	reduces random variation
Hysteresis	prevents rapid switching near a boundary
Rate limiting	reduces abrupt motor or servo commands
Timeouts	detects missing signals or behavior that lasts too long
Safe startup	prevents unexpected motion after upload or reset
Mechanical securing	prevents intermittent wires and shifting components
Repeated tests	reveals inconsistent behavior

39. Real-System Reliability Questions



Automatic door: What should happen if the presence sensor remains active, the doorway is blocked, or power returns after an outage?

Actual photograph reused from the instructor's earlier ENGR 1105 media library.



Robotic vacuum: What states are needed for normal cleaning, obstacle detection, low battery, wheel stall, and user stop?

Actual photograph reused from the instructor's earlier ENGR 1105 media library.

40. Safety Review Before Full Integration

Final Project Safety Review

Before full-power testing:

1. verify actuator and supply voltage;
2. confirm driver current capability;
3. confirm common ground and polarity;
4. begin with stopped or limited actuator commands;
5. secure batteries, wheels, servo horns, and wires;
6. keep moving parts clear of people and cables;
7. define an immediate stop method;
8. know which fault requires power disconnection.

41. Documentation as Part of Reliability

Required documentation should include:

- project purpose and requirements;
- block diagram and pin table;
- wiring diagram or clear connection table;
- final code with comments;
- state diagram or behavior table;
- test matrix and observed results;
- known limitations and future improvements.

Another Person Should Be Able to Understand the System

Good documentation allows a teammate, instructor, or future student to trace the design, repeat the test, and identify where a failure may occur.

42. Week 6 Integration Readiness Checklist

Check	Readiness item
☐	Purpose and three to five testable requirements are written.
☐	Block diagram separates information and power paths.
☐	Pin assignments and subsystem interfaces are documented.
☐	Power sources and current paths are verified.
☐	Each sensor and output works independently.
☐	Startup and fault outputs are safe.
☐	States and transitions are defined.
☐	Test matrix includes normal, boundary, and failure conditions.
☐	Mechanical parts and wiring are secured.

43. Week 6 Summary

Core Ideas ---

- A project idea becomes an engineering design when it includes testable requirements and constraints.
- A block diagram separates sensing, control, output interfaces, actuators, and power.
- Interfaces must specify signals, pins, units, voltage levels, and expected behavior.
- A power budget considers startup and worst-case actuator current.
- Incremental integration preserves a known-good baseline and reduces troubleshooting time.
- State-based logic organizes complex behavior into named operating conditions.
- Feedback uses measured results in the next decision.
- Hysteresis, validation, timeouts, and repeated confirmation improve reliability.
- `millis()` supports responsive nonblocking timing.
- Fault states and safe defaults prevent invalid inputs from causing unsafe motion.
- Testing should cover units, subsystems, interfaces, the complete system, boundaries, and failure conditions.
- Good troubleshooting uses evidence, isolates one layer, changes one item, and retests.

Lecture 6.2 Exit Questions ---

1. What is the difference between an open-loop command and feedback behavior?
2. Why is a named FAULT state useful?
3. What is the benefit and cost of repeated confirmation?
4. Why can `delay()` make a project less responsive?
5. What four items belong in every test case?
6. Why should boundary and power-cycle behavior be tested?

Before Lab 6 ---

Bring the complete final-project hardware, the latest Arduino code, block diagram, pin table, power plan, state plan, and test matrix. Begin integration with the actuator stopped or at a safe position.